

# Process Management Guide — Webflow · Xano · Cloudflare · Shopify

When you operate across independent platforms, the most dangerous move is letting one of them be both your **source of truth** and your **runtime dependency**. The June 3 2026 global Shopify outage — and Klaviyo's failure the next day — are the case studies. This defines the role each platform plays, how data flows between them, and the runbook that keeps you running when a platform you don't control goes dark.

**Audience:** Operations, platform engineering, and business stakeholders running a multi-system commerce stack

**Scope:** Public guide. No secrets, credentials, or source code. Process, architecture, and automation recommendations only.

**Date:** 2026-06-08

---

## 0. TL;DR

When you operate across **multiple independent platforms**, the most dangerous decision you can make is to let any one of them be both your **source of truth** and your **runtime dependency**. The June 3, 2026 global Shopify outage — where storefronts worldwide resolved to *"This store does not exist"* — is the case study that proves it. And the very next day, **June 4, Klaviyo went down too** (logins, APIs, event ingestion, and campaign/flow sends), proving the broader point: *this is not a Shopify problem — every platform you don't control will eventually have a bad day, sometimes back-to-back*. This guide defines the roles each platform should play, how data should move between them (the "ORM / Content / Data Layer" split), and the operating process that keeps you running when a platform you don't control goes dark.

**One-line recommendation:** *Shopify is a channel, not your database. Xano is the source of truth, Cloudflare is the resilient runtime, Webflow is the presentation layer, and Shopify is one of several destinations that data flows **to** — never the thing everything else flows **from**.*

---

1. THE FOUR PLATFORMS AND THE ROLE EACH SHOULD PLAY

| PLATFORM   | CORRECT ROLE                                                                                                                                         | WHAT IT MUST NOT BECOME                                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Shopify    | Commerce <b>channel</b> — catalog, checkout, payments, POS. A <i>destination</i> and a transaction engine.                                           | The single source of truth for your catalog, customers, or content. A hard runtime dependency for pages that must stay up. |
| Xano       | <b>Data layer / source of truth</b> — the relational system of record for catalog, customers, consent, orders, and sync state.                       | A passive cache. If Xano only mirrors Shopify, you have no independent truth to recover from.                              |
| Cloudflare | <b>Resilient runtime</b> — Worker that routes requests, verifies identity, enforces consent, caches reads at the edge, and absorbs upstream outages. | A thin proxy that fails the moment Shopify fails.                                                                          |
| Webflow    | <b>Content / presentation layer</b> — marketing pages, CMS content, and the Designer Extension your team operates.                                   | A second uncontrolled source of truth that drifts from Xano.                                                               |

The recurring failure mode across every team that has been burned: they let **Shopify be the database**. Then the day Shopify is unreachable, *everything* — storefront, content, internal tooling, analytics — goes down with it. The architecture below is designed so that a total Shopify outage degrades you to "checkout temporarily unavailable" instead of "business does not exist."

2. CASE STUDY — THE JUNE 3, 2026 GLOBAL SHOPIFY OUTAGE

2.1 What happened

On the morning of **June 3, 2026**, Shopify suffered a **global outage** affecting merchants and customers worldwide. Per Shopify's public status updates (reported by PYMNTS):

| TIME (EDT) | STATUS                                                                                                                     |
|------------|----------------------------------------------------------------------------------------------------------------------------|
| 9:27 a.m.  | Shopify announced issues across one or more functions: <b>admin, checkout, storefronts, Retail POS, and support access</b> |
| 10:37 a.m. | Identified the problem; reported recovery from mitigation efforts                                                          |
| 11:31 a.m. | Declared resolved                                                                                                          |
| 3:13 p.m.  | Final note: <i>"This issue has been resolved, and we are continuing to monitor"</i>                                        |

The **officially reported window was roughly two hours**, with monitoring continuing into the afternoon. Many merchants reported **degraded and intermittent impact lasting much of the day** — so plan for the worst case (a multi-hour to all-day disruption), not the press-release best case. DownDetector logged **3,000+ reports**, spiking before 9 a.m. EDT.

## 2.2 The error that made it worse

During the outage, visitors to affected stores saw:

**"This store does not exist"** — accompanied by a Shopify advertisement.

This is the part that turns a vendor incident into *your* reputational incident. The error doesn't say "temporarily down." It tells your customers your business **no longer exists**, and advertises the platform on your storefront's grave. Merchants publicly demanded Shopify swap it for an honest outage notice:

*"Change the 'This store does not exist' page to a page that says 'We are experiencing an outage — we'll be back soon.' Having our stores resolve to that page is insane."*

## 2.3 The lessons (these drive every recommendation below)

1. **You do not control your platform's uptime, and you do not control its error page.** When Shopify is down, customers see Shopify's chosen message about *your* brand — and it can be actively damaging.
2. **A platform outage is a single point of failure if you let it be one.** If your homepage, your content, and your internal tools all depend on a live Shopify response, one outage takes out everything at once.

3. **"Resolved" for the vendor ≠ "recovered" for you.** Caches, webhooks, and queued events need to drain and reconcile after the platform returns. Budget for a recovery tail.
4. **Read availability and write availability are different problems.** You can keep *browsing* alive with cached/independent data far more easily than you can keep *checkout* alive — so protect them differently (Section 5).

## 2.4 The next day — Klaviyo (June 4, 2026)

Less than 24 hours later, **Klaviyo** — a marketing/CDP platform in the *event-ingestion and messaging* layer — had its own disruption. Per status reporting, the impact spanned **access to klaviyo.com, the APIs, event ingestion, and subscriptions**: customers could be **unable to log in, send campaigns or flows, or have their events processed**. (The exact incident window isn't fully detailed in public reporting; treat the duration as "multi-hour, plan for the worst.")

Why this matters for the architecture:

- **It's a different layer failing.** Shopify is the *commerce channel*; Klaviyo is a *destination for customer events*. Two different platforms, two different layers, two consecutive days. The lesson generalizes: **assume every external dependency will fail; design so none of them can take you down.**
- **Event ingestion failing is a silent data-loss risk.** If your app fires customer/order events straight at Klaviyo and Klaviyo is rejecting them, those events are **gone** unless you buffered them. This is exactly why cross-system events must be **queued, archived (R2), and replayable** (Section 4) rather than fired-and-forgotten — when Klaviyo recovers, you replay the backlog and lose nothing.
- **Marketing/consent events belong in your data layer first.** If an event matters (a signup, a consent change, a purchase), it should land in **Xano** as the system of record and be *projected* to Klaviyo/GA4/Adobe — never originate only inside a third-party platform that can drop it.

## 2.5 Sources

- [Shopify Resolves 2-Hour Outage Impacting Storefronts and Checkouts — PYMNTS](#)
  - [Ecommerce Outage Playbook: Survive Platform Downtime — Digital Applied](#)
  - [Klaviyo Status — Incident History](#)
-

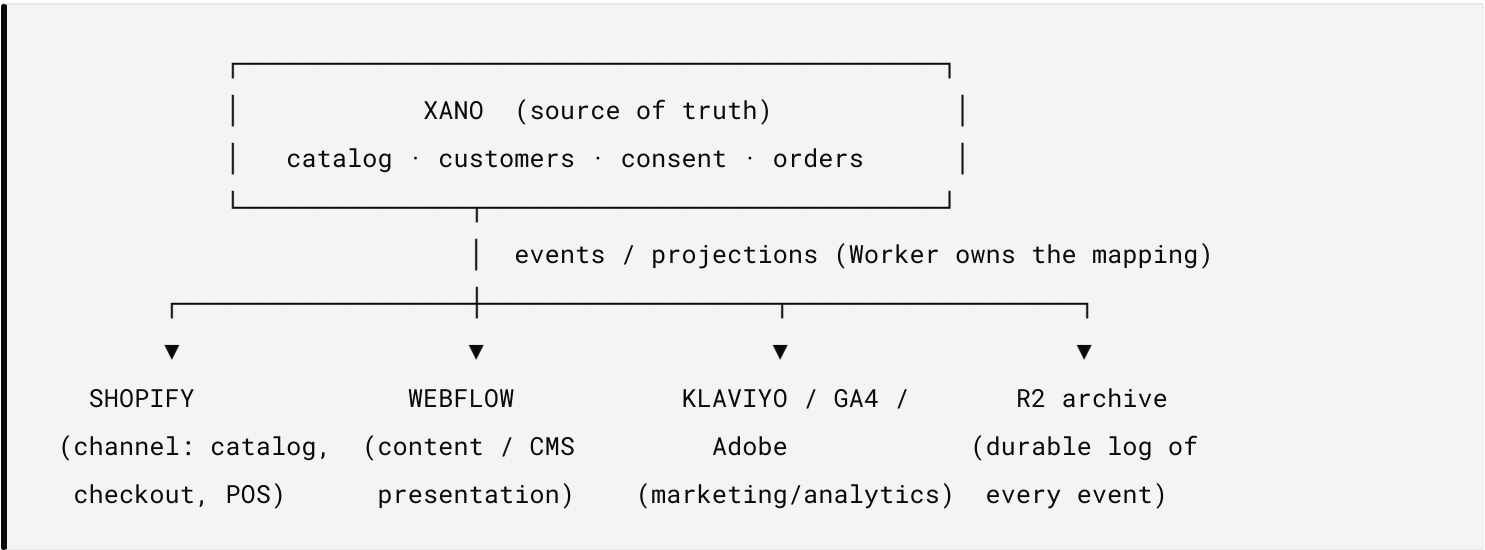
3. THE ORM / CONTENT / DATA LAYER SPLIT

"ORM" here doesn't mean a code library – it means the **object-relational mapping between systems**: how a product, customer, or order in one platform is represented and kept consistent in the others. Get this mapping wrong and every outage, schema change, or rate-limit becomes a data-integrity incident.

3.1 Three layers, three owners

| LAYER                            | OWNER                          | LIVES IN                                                                     | MAPPING RESPONSIBILITY                                                                                                                        |
|----------------------------------|--------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Data layer<br>(system of record) | Xano                           | Relational tables: catalog, variants, customers, consent, orders, sync state | Canonical IDs. Every external object maps back to a Xano row by a stable natural key.                                                         |
| ORM / projection layer           | Cloudflare Worker              | Stateless transforms + edge cache (KV)                                       | Translates a Xano record $\rightleftharpoons$ Shopify object $\rightleftharpoons$ Webflow CMS item. Owns idempotency and conflict resolution. |
| Content / presentation layer     | Webflow (+ Shopify storefront) | CMS items, pages, product display                                            | Renders what the projection layer publishes. Never the origin of truth.                                                                       |

3.2 Direction of flow (the rule that survives outages)



**The invariant:** data flows **out of** Xano to every channel. No channel is allowed to be the only place a fact exists. Shopify receives the catalog and returns orders; it does not *own* the

catalog. When Shopify disappears for two hours, the catalog still exists in Xano, the content still renders from Webflow + edge cache, and orders captured during the gap reconcile when Shopify returns.

### 3.3 Mapping discipline (avoid the classic ORM bugs)

- **Stable natural keys**, not platform IDs, as the join key (e.g. SKU/handle), so a record survives being re-created in any one platform.
- **Idempotent upserts** keyed on `(natural_key, source)` so a retried webhook or a replayed event can't create duplicates.
- **One writer per field**. Decide which platform is authoritative for each field (price → Xano; published-state → Webflow; fulfillment-state → Shopify) and never let two systems write the same field.
- **Versioned records** with `updated_at` / monotonic version so conflict resolution is deterministic (last-writer-wins by version, not by wall clock).

---

## 4. AUTOMATION RECOMMENDATION – EVENT-DRIVEN, NOT PLATFORM-COUPLED

---

### 4.1 Replace "live pull" with "cached read + async write"

The outage-resilient pattern, and the one this stack already implements:

| PATTERN                     | READ PATH                                                                         | WRITE PATH                                                                                                   |
|-----------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| ✗ <b>Fragile</b><br>(don't) | Page calls Shopify live on every request                                          | Page writes directly to Shopify, fails if Shopify is down                                                    |
| ✓ <b>Resilient</b><br>(do)  | Page reads from Xano / Cloudflare edge cache; Shopify is refreshed asynchronously | Writes go to a <b>queue</b> ; the Worker delivers to Shopify with retries, dead-letter, and circuit breaking |

### 4.2 The delivery guarantees that make an outage survivable

These are operating defaults, not aspirations:

- **Queue every cross-system change** ( `integration_queue` in Xano) instead of calling platforms inline. An outage just means the queue gets deeper, not that work is lost.

- **Per-priority retry with backoff** — critical inventory retries hardest, batch analytics softest:

| PRIORITY      | MAX RETRIES | BACKOFF                    | DEAD-LETTER AFTER |
|---------------|-------------|----------------------------|-------------------|
| 1 — inventory | 5           | 10s · 30s · 2m · 10m · 30m | 30 min            |
| 2 — orders    | 4           | 30s · 2m · 10m · 1h        | 1 hour            |
| 3 — customers | 3           | 2m · 15m · 1h              | 1 hour            |
| 5 — analytics | 2           | 1h · 6h                    | 6 hours           |

- **Circuit breaker per destination** — if Shopify fails 5× in 10 minutes, stop hammering it for a cooldown, then auto-probe. During the June 3 outage this is what stops you from burning your rate limit and your logs against a dead endpoint, and lets you recover cleanly the instant it returns.
- **Dead-letter queue, never discard** — failed events are stored in Xano, archived in R2, and alerted on. After recovery you replay them in order. *Nothing captured during the outage is lost.*
- **A durable event archive (R2)** so you always have an independent, append-only record of what happened — including everything that piled up while a platform was down.

### 4.3 What stays on a timer

Cron is reduced to a **safety net**, not the primary mover: token refresh, dead-letter sweep, and a catch-up reconciliation pass that compares Xano ⇔ each channel and heals drift introduced during an outage.

## 5. OUTAGE RUNBOOK — WHAT TO DO WHEN A PLATFORM GOES DARK

This is the operational core of "process management." Keep it short, rehearsed, and owned.

### 5.1 Detection (minutes 0–5)

- **Watch external signal, not just your own dashboards.** A platform outage won't show as *your* error — it shows as upstream 5xx/timeouts and a DownDetector spike. Subscribe to each platform's status page.

- **Confirm scope:** is it read, write, or both? Storefront, admin, checkout, POS, or API? (June 3 hit *all* of them.)
- **Confirm blast radius:** which of your surfaces actually depend on the down platform right now?

## 5.2 Containment (minutes 5–20)

| IF DOWN...                         | DO                                                                                                                                                                                                                   |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Shopify storefront</b>          | Serve cached catalog/content from Cloudflare + Webflow. Replace any Shopify-rendered surface you control with <b>your own honest outage banner</b> — never let customers land on <i>"This store does not exist."</i> |
| <b>Shopify checkout / payments</b> | Show a clear "checkout temporarily unavailable — we'll email you / try again shortly" state. Capture intent (email, cart) into Xano so you can recover the sale. Do <b>not</b> silently fail the buy button.         |
| <b>Shopify admin / API</b>         | Stop inline writes; let the queue absorb them. Verify the circuit breaker has opened.                                                                                                                                |
| <b>Xano (source of truth)</b>      | This is your most severe case — failover/read-replica strategy and edge cache become primary. Halt writes that can't be reconciled.                                                                                  |
| <b>Webflow</b>                     | Serve last-published static content from cache/CDN; pause CMS publishes.                                                                                                                                             |
| <b>Cloudflare</b>                  | Lean on edge cache + status comms; this is the platform whose job is to <i>be</i> the resilient layer, so its own incidents are highest-severity.                                                                    |

## 5.3 Communicate (in parallel)

- Post your own status (status page / banner / social) **before** customers post for you. "We're aware, here's what works, here's what doesn't, here's the ETA."
- Brief support with the known facts and the workaround.

## 5.4 Recovery (when the platform returns)

1. **Don't trust "resolved."** Probe with a single canary request before reopening the floodgates.
2. **Let circuit breakers auto-close;** watch error rate as traffic resumes.
3. **Replay the dead-letter queue** in priority order.



4. **Run reconciliation** (Xano ⇌ Shopify ⇌ Webflow) to heal any drift — especially orders/inventory that moved during the gap.
5. **Remove your outage banners** only after reconciliation is clean.

### 5.5 Post-incident (within 48h)

Short, blameless write-up: timeline, customer impact, what the cache/queue saved you, what didn't degrade gracefully, and one or two concrete hardening actions. File it; review it next time.

### 6. ROLES & RESPONSIBILITIES (RACI-LITE)

| ACTIVITY                                               | PLATFORM ENG | OPS / SUPPORT       | BUSINESS OWNER |
|--------------------------------------------------------|--------------|---------------------|----------------|
| Schema / mapping changes (Xano source of truth)        | <b>Owns</b>  | Informed            | Consulted      |
| Channel config (enable/disable Shopify, GA4, Adobe...) | Consulted    | <b>Owns</b>         | Informed       |
| Deploys (Worker / extension / app)                     | <b>Owns</b>  | Informed            | Informed       |
| Outage detection & runbook execution                   | <b>Owns</b>  | <b>Owns</b> (comms) | Informed       |
| Customer communication during incident                 | Consulted    | <b>Owns</b>         | Accountable    |
| Post-incident review                                   | <b>Owns</b>  | Contributes         | Accountable    |

**Principle (from the security architecture):** each platform is independently controllable. Disabling a misbehaving channel is a **config toggle, not a code change** — so Ops can contain an incident in seconds without waiting on a deploy.

### 7. CHANGE & DEPLOY PROCESS (SUMMARY)

The stack has **three independently deployable units** — they ship separately so a change to one can't take down the others:

1. **Cloudflare Worker** — the runtime/ORM-projection layer (data plane).
2. **Webflow Designer Extension** — the operator interface (validate before publish).
3. **Public spec / docs** — stakeholder-facing, **no source code, no secrets**.

Change-management rules:

- **One source of truth per fact** — changes to canonical data go through Xano; channels are projected, never hand-edited into drift.
  - **Backward-compatible first** — add new endpoints/aliases before redirecting traffic; run old and new paths in parallel, then retire.
  - **Every config change is authenticated and logged** with before/after, so any change can be audited and reversed.
  - **Never commit secrets**; credentials live only in the encrypted per-tenant store, never in code or docs.
- 

## 8. CHECKLIST — IS YOUR STACK JUNE-3-PROOF?

---

- ☐ **Xano is the source of truth** — every catalog/customer fact exists independently of Shopify.
  - ☐ **Storefront reads survive a Shopify outage** from Xano + edge cache.
  - ☐ **You control the outage page** — customers never see *"This store does not exist."*
  - ☐ **Checkout fails loudly and captures intent** instead of silently dropping sales.
  - ☐ **All cross-system writes go through a queue** with retry + dead-letter + circuit breaker.
  - ☐ **Marketing/event sends (Klaviyo, GA4, Adobe) are buffered**, not fired-and-forgotten — a downstream outage backs up the queue instead of losing events.
  - ☐ **Nothing is discarded on failure** — dead letters are stored, archived, and replayable.
  - ☐ **A reconciliation pass** can heal Xano  $\rightleftharpoons$  Shopify  $\rightleftharpoons$  Webflow drift after recovery.
  - ☐ **Status-page subscriptions** for all four platforms feed your alerting.
  - ☐ **The outage runbook is written, owned, and rehearsed** — not improvised at 9:27 a.m.
  - ☐ **One writer per field**; mapping uses stable natural keys and idempotent upserts.
-

*If you can check every box, a global Shopify outage is a degraded hour, not an existential one. That is the entire point of running four platforms instead of one.*